CPU Scheduling Algorithms in Modern Operating Systems

CSE 4501 : Operating Systems

Assigned to :

Saadman Sakib
ID: 210042165

Program : BSc. Engg. SWE
Department of Computer Science and Engineering
Islamic University of Technology

1. Introduction

CPU scheduling is a mechanism in operating systems that determines which processes are assigned to the processor for execution. Efficient scheduling increases **CPU utilization**, reduces **waiting time**, and enhances **user responsiveness** in both desktop and server systems.

Key concepts:

- **CPU Burst:** Time a process spends executing on the CPU before needing I/O.
- **Turnaround Time:** Total time from process submission to completion.
- **Waiting Time:** Total time a process waits in the ready queue.
- **Response Time:** Time from submission until the first response is produced.

Scheduling can be:

- **Preemptive:** OS can interrupt and switch tasks.
- **Non-preemptive:** Task runs until completion or blocking.

2. Understanding Scheduling Algorithms

2.1 First Come First Serve (FCFS)

- **Type:** Non-preemptive
- **Concept:** Processes are scheduled in order of arrival.
- **Diagram:**

  Linear timeline (P1 → P2 → P3…)

- **Pros:** Simple to implement.
- **Cons:** Poor average waiting time, risk of convoy effect.
- **Best for:** Batch systems.

2.2 Shortest Job First (SJF)

- **Type:** Non-preemptive or preemptive (SRTF)
- **Concept:** Processes with shortest CPU burst time get priority.
- **Diagram:** Processes Sorted from shortest to longest :

  P2, P1, P3

  P2 → P1 → P3

- **Pros:** Optimal average waiting time.
- **Cons:** Difficult to predict burst time; lead to starvation.
- **Best for:** Predictable workloads.

2.3 Priority Scheduling

- **Type:** Both
- **Concept:** Each process has a priority; lower values = higher priority.
- **Diagram:**

    P3 (priority 1) → P1 (priority 2) → P2 (priority 3)…

- **Pros:** Meets business or system priority needs.
- **Cons:** Starvation of low-priority tasks.
- **Best for:** Real-time systems with critical tasks.

2.4 Round Robin (RR)

- **Type:** Preemptive
- **Concept:** Each process gets a time quantum (e.g., 4ms) in cyclic order.
- **Diagram:**

    P1 → P2 → P3 → P1 → P2…

- **Pros:** Fair and responsive.
- **Cons:** High context switching overhead.
- **Best for:** Time-sharing systems.

2.5 Multilevel Queue Scheduling

- **Type:** Non-preemptive (static queue separation)
- **Concept:** Processes divided into queues by type (interactive, batch).

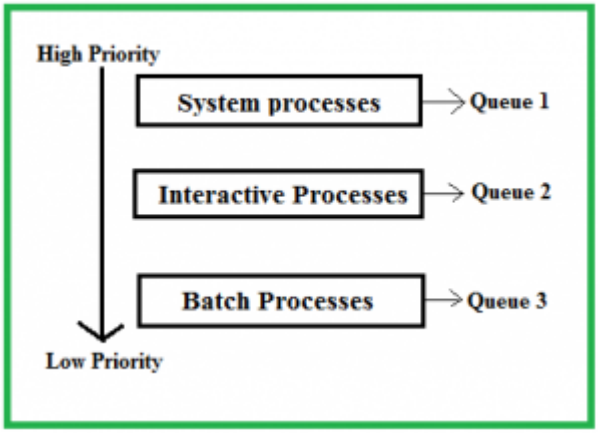


Fig 2.1 : MLQ Scheduling with 3 levels (ref : https://www.geeksforgeeks.org/)

- **Pros:** Good for priority segregation.
- **Cons:** Inflexible.
- **Best for:** Systems with fixed task classes.

2.6 Multilevel Feedback-Queue

- **Type:** Preemptive
- **Concept:** Processes can move between queues based on behavior.

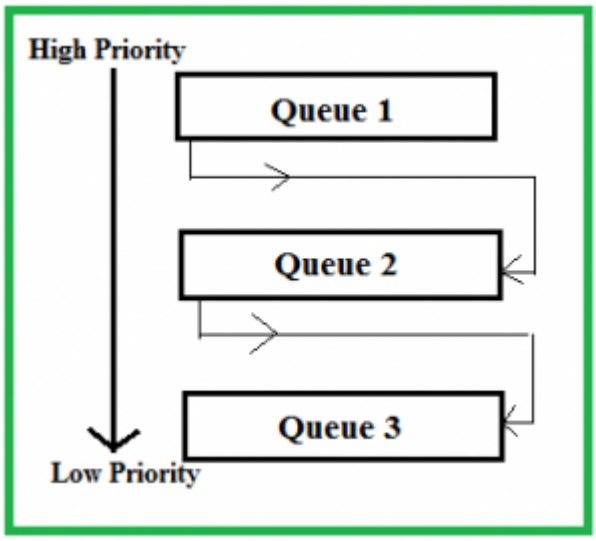


Fig 2.2 : MLFQ Scheduling with 3 levels (ref : https://www.geeksforgeeks.org/)

- **Pros:** Adaptive, balances responsiveness and throughput.
- **Cons:** Complex to tune.
- **Best for:** General-purpose OS (e.g., Windows, Linux).

3. Algorithm Performance Evaluation

Selected Algorithms: **FCFS, SJF, Round Robin**

Hypothetical Processes:

Process	Arrival Time AT (ms)	Burst Time BT (ms)
P1	0	5
P2	1	3
P3	2	8
P4	3	6

3.1 Comparison Table Preparation :

3.1.1 Manual Calculations

- FCFS (First Come First Serve) – Non-preemptive

  Order: P1 → P2 → P3 → P4
  (All processes assumed to arrive in increasing order)

Process	Start Time ms	Completion Time (CT) ms	Turnaround Time (TT = CT - AT) ms	Waiting Time (WT = TT - BT) ms
P1	0	5	5	0
P2	5	8	7	4
P3	8	16	14	6
P4	16	22	19	13

Avg Waiting Time: (0 + 4 + 6 + 13) / 4 = 5.75 ms
Avg Turnaround Time: (5 + 7 + 14 + 19) / 4 = 11.25 ms

- SJF (Shortest Job First) – Non-preemptive

  Sorted by Burst Time (when available):

  At time 0: P1 (5)
  At time 5: P2 (3), P3 (8), P4 (6) → Choose P2
  Then P4 (6), then P3 (8)

  Order: P1 → P2 → P4 → P3

Process	Start Time ms	Completion Time (CT) ms	Turnaround Time (TT = CT - AT) ms	Waiting Time (WT = TT - BT) ms
P1	0	5	5	0
P2	5	8	7	4
P3	8	14	11	5
P4	14	22	20	12

Avg Waiting Time: (0 + 4 + 5 + 12) / 4 = 5.25 ms
Avg Turnaround Time: (5 + 7 + 11 + 20) / 4 = 10.75 ms

- **Round Robin (RR), Quantum = 4ms**

  Order of Execution:

  Time 0: P1(5) → P1(4), remaining = 1

  Time 4: P2(3) → P2(3), done

  Time 7: P3(8) → P3(4), remaining = 4

  Time 11: P4(6) → P4(4), remaining = 2

  Time 15: P1(1), done

  Time 16: P3(4) → P3(4), done

  Time 20: P4(2) → P4(2), done

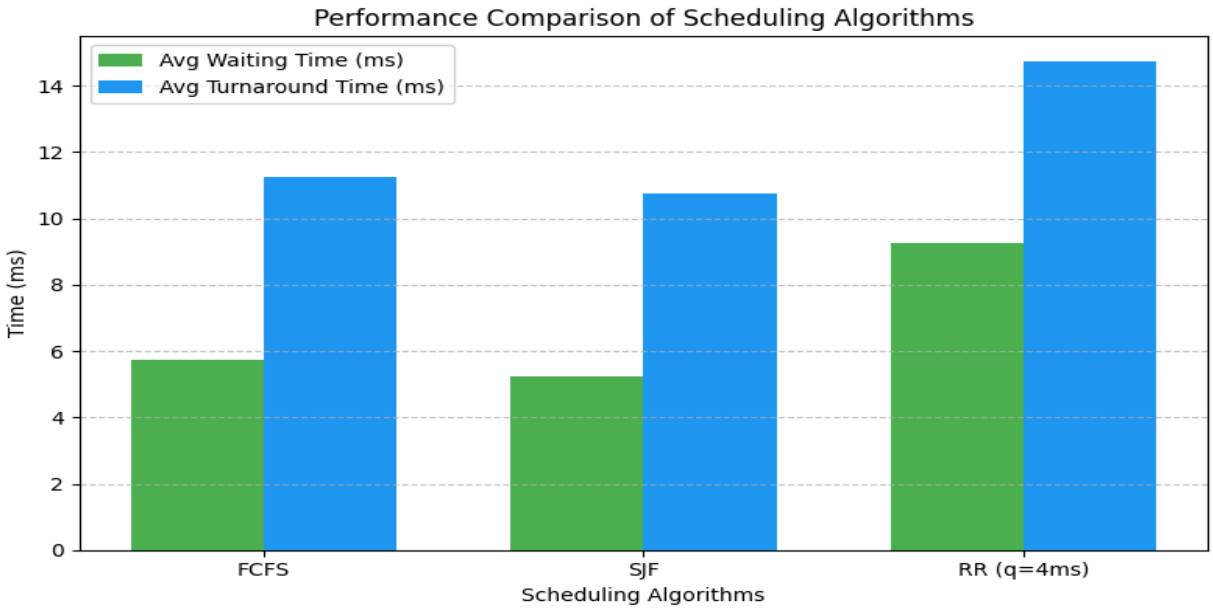
Process	Completion Time (CT) ms	Turnaround Time (TT = CT - AT) ms	Waiting Time (WT = TT - BT) ms
P1	16	16 - 0 = 16	16 - 5 =11
P2	7	7 - 1 = 6	6 - 3 = 3
P3	20	20 - 2 = 18	18 - 8 = 10
P4	22	22 - 3 = 19	19 - 6 = 13

Avg Waiting Time: (11 + 3 + 10 + 13) / 4 = 9.25 ms
Avg Turnaround Time: (16 + 6 + 18 + 19) / 4 = 14.75 ms

3.1.2 Final Comparison Table

Metric	FCFS	SJF	Round Robin (q=4ms)
Avg Waiting Time (ms)	5.75	5.25	9.25
Avg Turnaround Time(ms)	11.25	10.75	14.75
CPU Utilization	High	High	Medium
Fairness	Low	Low	High

3.2 Graph :



4. Analytical Application

Impact of Preemption –

- Responsiveness: Improves user experience (e.g. Round Robin).
- Fairness: Prevents long wait times for short tasks.

Starvation –

- Occurs in SJF and Priority Scheduling when short/high-priority tasks dominate.
- Solution: Aging—gradually increasing the priority of waiting processes.

5. Real-World OS Case Study

Linux:

- Uses Completely Fair Scheduler (CFS).
- Based on weighted fair queuing.
- Focuses on interactivity and fairness.
- Adjusts priorities dynamically.

Windows (e.g., Windows 11):

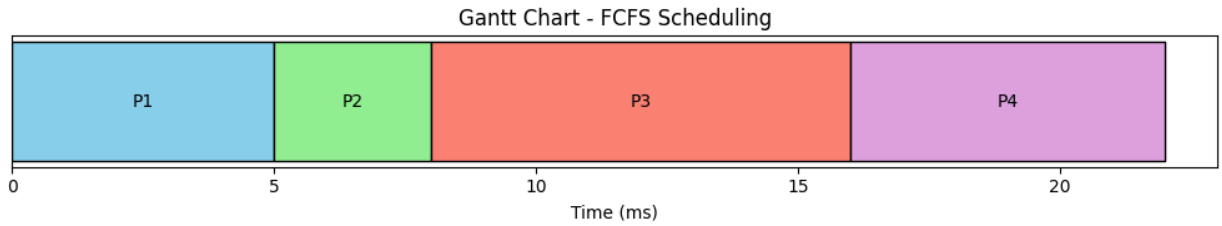
- Uses Multilevel Feedback Queue with priority boosting.
- Prioritizes foreground apps.
- Handles context switches efficiently.

Desktop vs. Server:

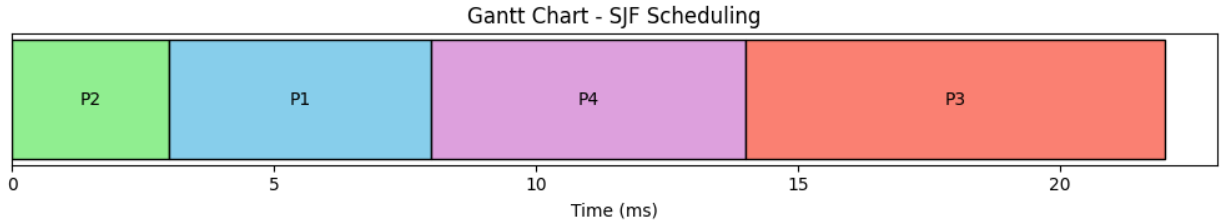
- Desktop: Emphasizes responsiveness.
- Server: Focuses on throughput and predictability.

6. Visualization Task: Gantt Charts

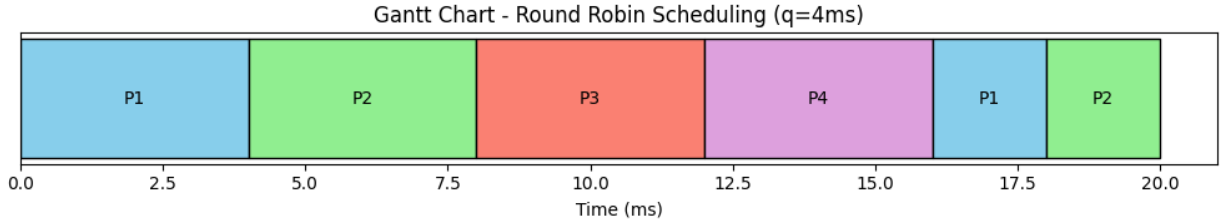
6.1 Gantt Chart : FCFS



6.2 Gantt Chart : SJF



6.3 Gantt Chart : Round Robin Scheduling



7. Critical Thinking Scenario

Scenario: Real-Time Embedded System (e.g., Pacemaker)

- Chosen Algorithm: Fixed Priority Preemptive Scheduling
- Justification:
    - Deterministic execution guarantees.
    - Low latency (Quick Response to critical events)
    - Simplicity & Efficiency
    - Well Supported in Real-Time OS (like FreeRTOS, VxWorks, and RTEMS use this in deadline-strict scenarios)

In a pacemaker:

- High-priority task: Detecting abnormal heart rhythms.
- Low-priority task: Logging data to memory.

If both tasks are scheduled and a high-priority task then becomes ready, the system **immediately preempts** the logging process to stabilize the heartbeat—which ensures patient safety.

8. Conclusion

CPU scheduling is one of the basic parameters of OS performance. By comparing different algorithms, it becomes clear that no best solution exists—each algorithm has trade-offs between **efficiency**, **fairness**, and **responsiveness**. Real-world OSs like Linux and Windows adopt hybrid or dynamic models to afford varied workloads. Understanding these mechanisms let OS developers make systems more responsive, and resource-efficient.